

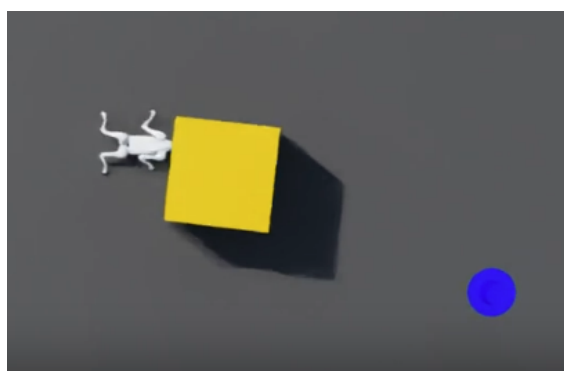
机器人学导论2026大作业

作业要求

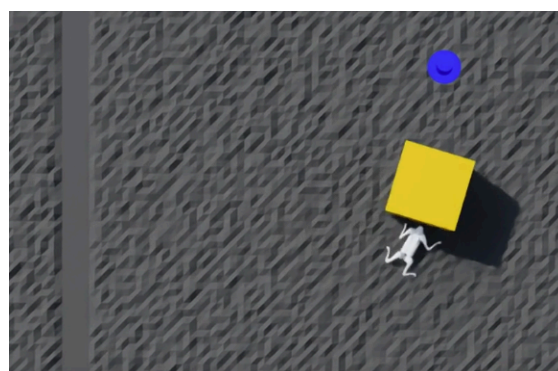
在本次作业中，请选择一个给定场景，在Isaac Lab中构建该场景，并训练机器人完成场景对应的任务，最终形成论文式的实验报告。实验报告需要包括摘要、引言、相关工作、问题背景（Preliminary）、方法（包括具体场景设计）、实验、总结等部分，每个部分的具体写作内容与写作方式可以参考组内的论文发表<https://lamda-rl.nju.edu.cn/papers.html>。

候选场景

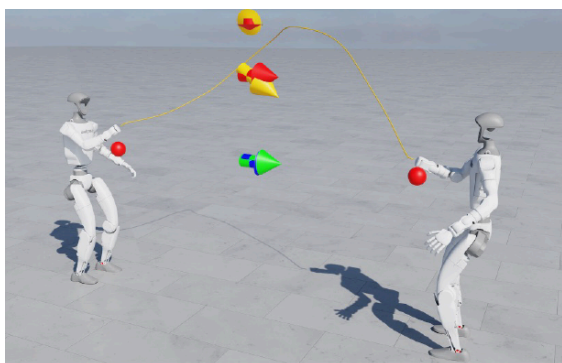
- 机器狗运动（奔跑）至目标点
- 人形机器人运动（奔跑）至目标点
- 无人机飞行至目标点
- 机器狗运动至目标点，非平地（例如斜坡，凹凸不平等）
- 机器狗经过地面上固定的矩形框后（从框中钻过）运动至目标点
- 双人形机器人甩绳
- 机器狗推箱子至目标点
- 机器狗追踪运动目标



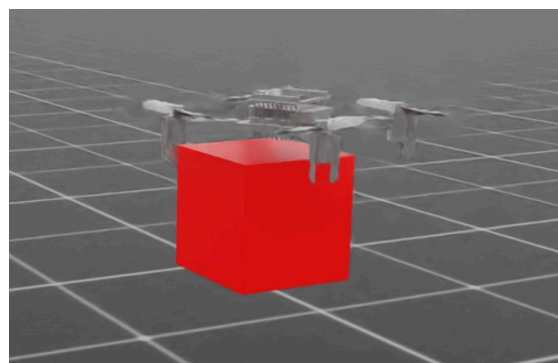
机器狗推箱子至目标点



机器狗在非平地中推箱子至目标点



双人形机器人甩绳



无人机飞行至目标点

所有场景的构建可以参考文档

<https://docs.robotsfan.com/isaacsim/source/overview/environments.html>，其中许多场景已经包括在其中。在我们所给定的场景的基础上添加额外内容，形成更丰富、更复杂的合理场景同样是允许的，并且可能获得更高的分数。

提交内容

作业请提交一个ZIP压缩文档，其中包括

- 实验报告PDF文件
- 完整的可运行代码
- 在Isaac Lab中训练完毕的智能体在对应场景中完成任务的视频

分数组成包括以下内容

- 场景构建
- 实验报告
- 策略训练成果

都是按步骤给分，因此，请尽你所能的完成更多的内容，同时，尽量不要放弃。即使策略训练未成功，成功构建场景并完成实验报告的绝大部分内容也是能获得大部分分数的。

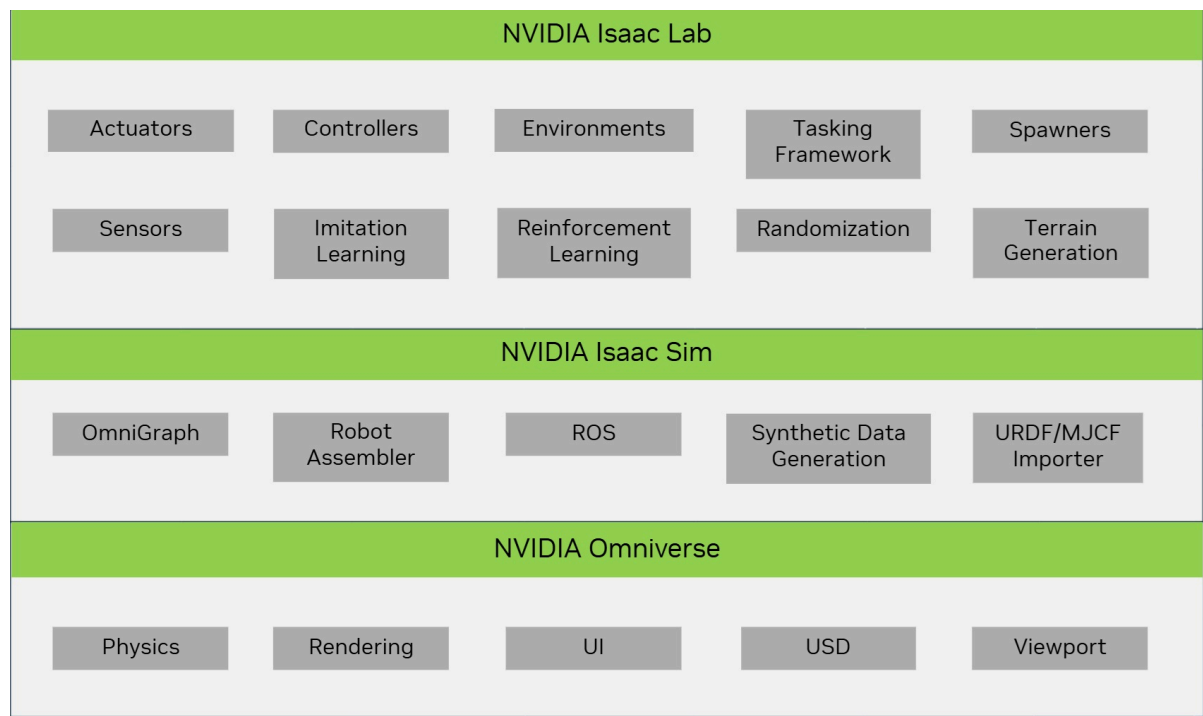
Isaac Lab 简明教程

原始参考资料，推荐同学们直接查阅

Isaac Lab 官方文档 <https://isaac-sim.github.io/IsaacLab/main/index.html>

Isaac Lab 代码仓库 <https://github.com/isaac-sim/IsaacLab>

Isaac Lab Ecosystem



Isaac Lab Tasks



运行环境配置

系统要求

Isaac Lab 基于 NVIDIA Isaac Sim 和 NVIDIA Omniverse 构建，对硬件有一定要求，官方推荐的配置是

配置项	建议
操作系统	Ubuntu 22.04 (Linux x64) 或 Windows 11
内存	≥ 32 GB
显存	≥ 16 GB (需 NVIDIA GPU)
Python 版本	3.11 (对应 Isaac Sim 5.X)

快速安装

推荐安装方式：**pip 安装 Isaac Sim + 源码安装 Isaac Lab**

```
# 1. 创建虚拟环境
conda create --name isaacrab python==3.11
conda activate isaacrab

# 2. 通过pip安装Isaac Sim
pip install "isaacsim[all,extscache]==5.1.0" --extra-index-url https://pypi.nvidia.com

# 3. clone Isaac Lab源码
git clone https://github.com/isaac-sim/IsaacLab.git
cd IsaacLab

# 4. 安装Isaac Lab中的子package (可以根据实际需要选择对应的安装方式)
./isaacrab.sh --install none      # 通过脚本安装所有package, 不包含rl_gamess、rsl_rl等强化学习库
./isaacrab.sh --install rsl_rl    # 通过脚本安装所有package, 额外安装rsl_rl强化学习库
pip install -e source/isaacrab -e source/isaacrab_assets  # 只安装核心package和资产配置package
```

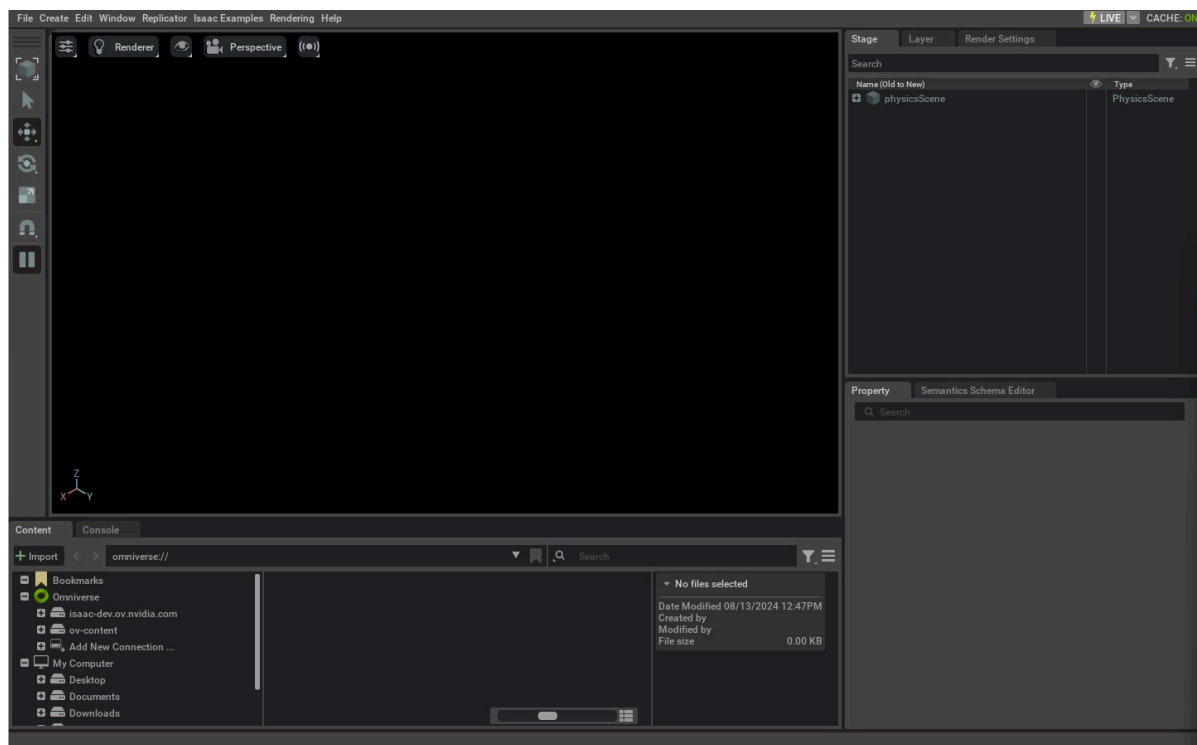
示例脚本运行

可以通过运行 Isaac Lab 中的 tutorial 脚本来验证是否配置成功

```
python scripts/tutorials/00_sim/create_empty.py
```

如果在个人 PC 上运行应该可以看到弹出的 Isaac Sim GUI 窗口

Isaac Sim GUI

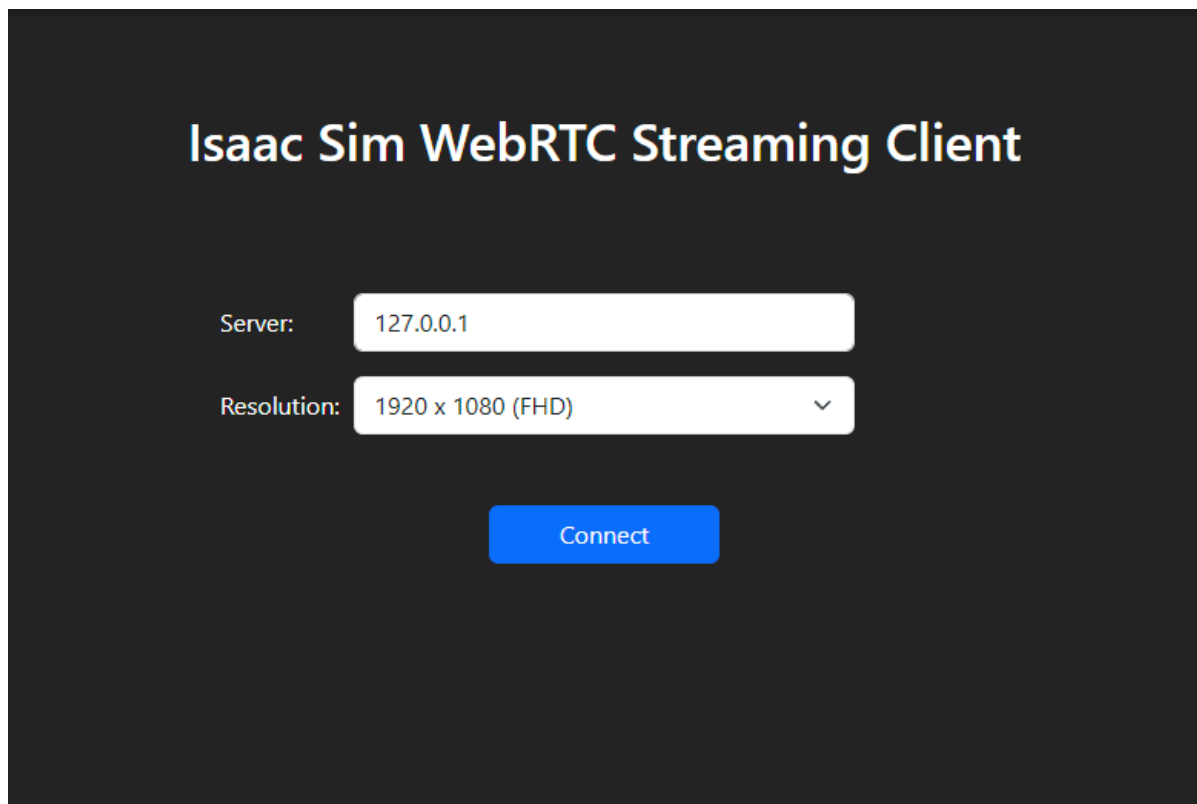


使用局域网内服务器的同学可以尝试使用 Isaac Sim WebRTC Streaming 来实现远程 GUI，在服务器端需要运行

```
python scripts/tutorials/00_sim/create_empty.py --livestrea  
m 2
```

随后在个人 PC 上打开 Isaac Sim WebRTC Streaming Client，在 **Server** 中输入服务器的 IP 地址连接即可

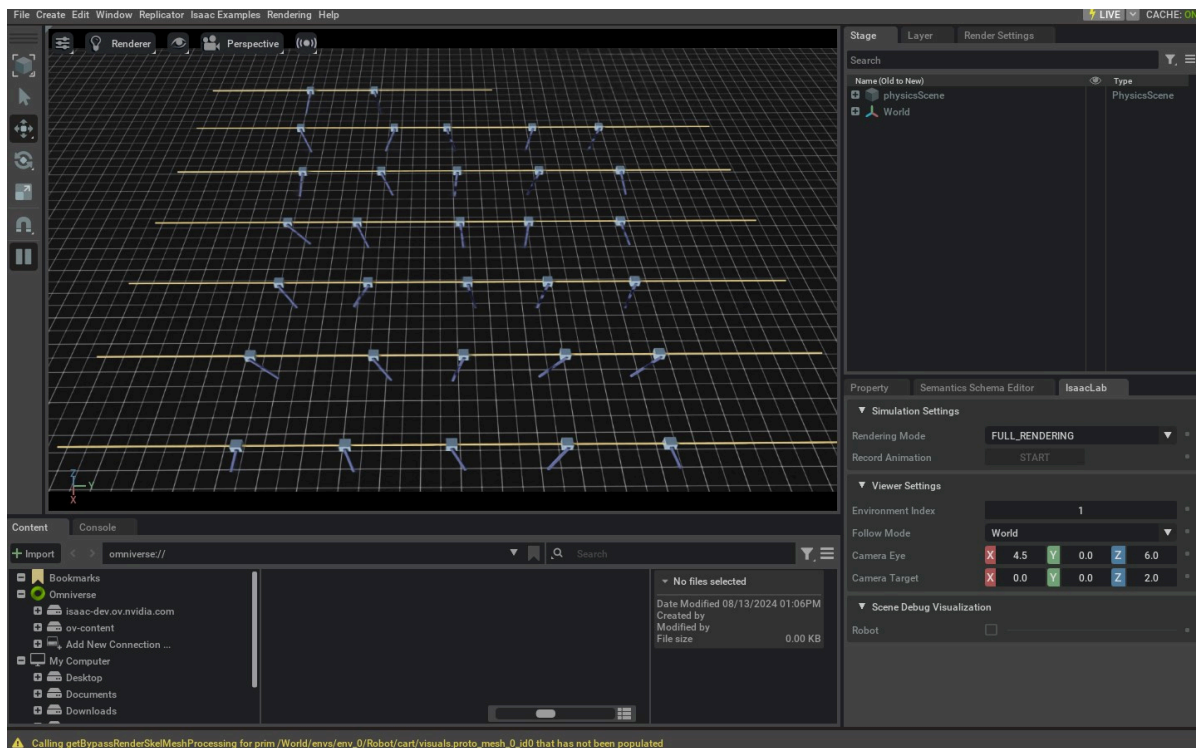
WebRTC Client



仿真环境搭建

这里通过 Isaac Lab 内置的倒立摆任务来介绍一下如何以模块化的方式组合出一个完整的强化学习环境

CartPole Environment



环境配置路径

`source/isaacsim_tasks/isaacsim_tasks/manager_based/classic/cartpole/cartpole_env_cfg.py`

整体结构

Cartpole 环境配置由 **Scene + 五个 Manager 配置 + 环境总配置** 组成：

```

cartpole_env_cfg.py
├─ CartpoleSceneCfg          # 场景：地面、灯光、机器人
├─ ActionsCfg               # 动作空间
├─ ObservationsCfg          # 观测空间
├─ EventCfg                 # 重置事件
├─ RewardsCfg               # 奖励项
├─ TerminationsCfg        # 终止条件
└─ CartpoleEnvCfg           # 汇总为 ManagerBasedRLEnvCfg
  
```

① 场景配置 `InteractiveSceneCfg`

```

@configclass
class CartpoleSceneCfg(InteractiveSceneCfg):
    # 地面（全局共享，不克隆）
    ground = AssetBaseCfg(
        prim_path="/World/ground",
        spawn=sim_utils.GroundPlaneCfg(size=(100.0, 100.
0))),
  
```

```

    )

    # 倒立摆机器人 (每个 env 克隆一份)
    robot: ArticulationCfg = CARTPOLE_CFG.replace(
        prim_path="{ENV_REGEX_NS}/Robot"
    )

    # 灯光
    dome_light = AssetBaseCfg(
        prim_path="/World/DomeLight",
        spawn=sim_utils.DomeLightCfg(color=(0.9, 0.9, 0.9),
        intensity=500.0),
    )

```

② 动作配置 **ActionsCfg**

```

@configclass
class ActionsCfg:
    joint_effort = mdp.JointEffortActionCfg(
        asset_name="robot",
        joint_names=["slider_to_cart"],
        scale=100.0,
    )

```

- 动作：对小车关节 `slider_to_cart` 施加力矩
- `scale=100.0`：将策略输出（通常 $\in [-1, 1]$ ）缩放到实际力矩范围
- `asset_name="robot"` 对应 Scene 中的 `robot` 字段

③ 观测配置 **ObservationsCfg**

```

@configclass
class ObservationsCfg:
    @configclass
    class PolicyCfg(ObsGroup):
        joint_pos_rel = ObsTerm(func=mdp.joint_pos_rel)
        joint_vel_rel = ObsTerm(func=mdp.joint_vel_rel)

```

```
def __post_init__(self):
    self.concatenate_terms = True # 拼接为单一向量

policy: PolicyCfg = PolicyCfg()
```

Cartpole 经典状态为 4 维：

$$\mathbf{s} = [x, \dot{x}, \theta, \dot{\theta}]^T$$

对应小车位置/速度、杆角度/角速度（相对默认姿态）。

④ 事件配置 `EventCfg`

```
@configclass
class EventCfg:
    reset_cart_position = EventTerm(
        func=mdp.reset_joints_by_offset,
        mode="reset",
        params={
            "asset_cfg": SceneEntityCfg("robot", joint_name
s=["slider_to_cart"]),
            "position_range": (-1.0, 1.0),
            "velocity_range": (-0.5, 0.5),
        },
    )
    reset_pole_position = EventTerm(
        func=mdp.reset_joints_by_offset,
        mode="reset",
        params={
            "asset_cfg": SceneEntityCfg("robot", joint_name
s=["cart_to_pole"]),
            "position_range": (-0.25 * math.pi, 0.25 * mat
h.pi),
            "velocity_range": (-0.25 * math.pi, 0.25 * mat
h.pi),
        },
    )
```

每次环境实例进行重置时，小车和杆的初始状态在指定范围内 **随机采样**，增加训练多样性。

⑤ 奖励配置 `RewardsCfg`

```
@configclass
class RewardsCfg:

    alive = RewTerm(func=mdp.is_alive, weight=1.0)
    terminating = RewTerm(func=mdp.is_terminated, weight=-2.0)
    pole_pos = RewTerm(
        func=mdp.joint_pos_target_l2,
        weight=-1.0,
        params={"asset_cfg": SceneEntityCfg("robot", joint_names=["cart_to_pole"]), "target": 0.0},
    )
    cart_vel = RewTerm(
        func=mdp.joint_vel_l1,
        weight=-0.01,
        params={"asset_cfg": SceneEntityCfg("robot", joint_names=["slider_to_cart"])}},
    )
    pole_vel = RewTerm(
        func=mdp.joint_vel_l1,
        weight=-0.005,
        params={"asset_cfg": SceneEntityCfg("robot", joint_names=["cart_to_pole"])}},
    )
```

总奖励为各项加权和：

$$r_t = \sum_i w_i \cdot f_i(s_t, a_t)$$

奖励项	权重	含义
<code>alive</code>	+1.0	存活奖励
<code>terminating</code>	-2.0	提前终止惩罚

奖励项	权重	含义
<code>pole_pos</code>	-1.0	杆角度偏离竖直方向的 L2 惩罚
<code>cart_vel</code>	-0.01	抑制小车速度
<code>pole_vel</code>	-0.005	抑制杆角速度

⑥ 终止配置 `TerminationsCfg`

```
@configclass
class TerminationsCfg:
    time_out = DoneTerm(func=mdp.time_out, time_out=True)
    cart_out_of_bounds = DoneTerm(
        func=mdp.joint_pos_out_of_manual_limit,
        params={"asset_cfg": SceneEntityCfg("robot", joint_
names=["slider_to_cart"]), "bounds": (-3.0, 3.0)},
    )
```

- `time_out=True` → Gymnasium 的 `truncated` (达到最大步数)
- 小车超出 $[-3, 3]$ m → `terminated` (任务失败)

⑦ 环境总配置 `CartpoleEnvCfg`

```
@configclass
class CartpoleEnvCfg(ManagerBasedRLEnvCfg):
    scene: CartpoleSceneCfg = CartpoleSceneCfg(num_envs=409
6, env_spacing=4.0)
    observations: ObservationsCfg = ObservationsCfg()
    actions: ActionsCfg = ActionsCfg()
    events: EventCfg = EventCfg()
    rewards: RewardsCfg = RewardsCfg()
    terminations: TerminationsCfg = TerminationsCfg()

    def __post_init__(self):
        self.decimation = 2 # 每 2 个物理步执行 1
        次策略步
        self.episode_length_s = 5 # 每回合 5 秒
        self.sim.dt = 1 / 120 # 物理步长 1/120 s
        self.sim.render_interval = self.decimation
```

控制频率：

$$f_{\text{control}} = \frac{1}{\text{decimation} \cdot dt} = \frac{1}{2 \times \frac{1}{120}} = 60 \text{ Hz}$$

⑧ 环境实例化

```
from isaacsim.app import AppLauncher
# △ 必须先启动 SimulationApp, 再导入 isaacsim、isaacsim 的其他相关模块
app_launcher = AppLauncher(args_cli)
simulation_app = app_launcher.app

from isaacsim.envs import ManagerBasedRLEnv
from isaacsim_tasks.manager_based.classic.cartpole.cartpole_env_cfg import CartpoleEnvCfg

env_cfg = CartpoleEnvCfg()
env_cfg.scene.num_envs = 16
env = ManagerBasedRLEnv(cfg=env_cfg)

observation, reward, terminated, truncated, info = env.step(action)
env.close()
simulation_app.close()
```

环境步进时大致进行了以下操作：

1. 接受传入的动作 `action`，并根据 `ActionsCfg` 对动作进行缩放等变换
2. 内循环 `decimation` 步，施加变换后的动作并推进物理仿真
3. 计算终止信号、奖励信号等输出
4. 对结束的环境进行 partial reset，重置这些环境实例上的状态，其他环境实例的状态保持不变
5. 计算观测信号
6. 返回五元组输出 `(observation, reward, terminated, truncated, info)`

其他环境示例

除了以上的倒立摆环境以外，大家还可以参考 `isaacsim_tasks` 中的其他任务或以下项目来进一步探索、理解仿真核心概念与环境设计的常用组件

1. Unitree RL Lab https://github.com/unitreerobotics/unitree_rl_lab
2. IsaacLab HARL <https://github.com/DIRECTLab/IsaacLab-HARL>